Université Bretagne Sud
IUT de Vannes
Département Informatique — 2026/2027

BUT 2 — R3.07. SQL dans un langage de programmation

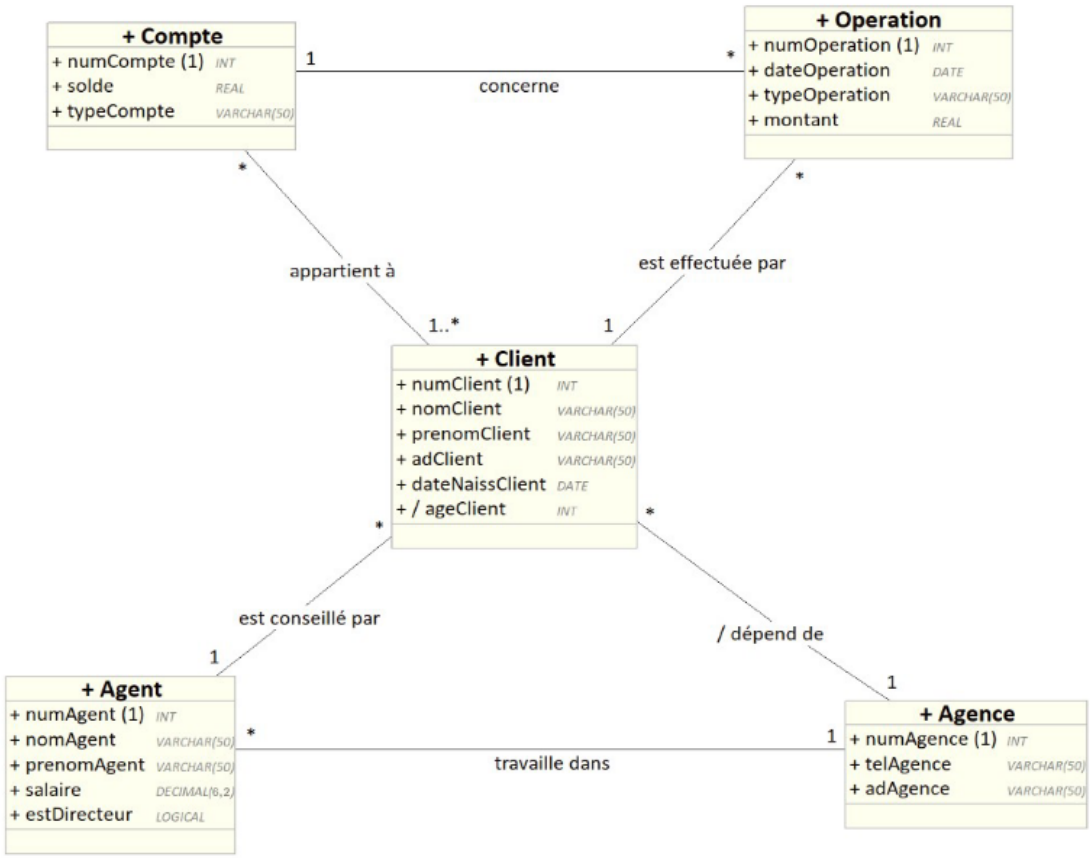
TD/TP 4

Java Persistence API (JPA) et patrons DAO/DTO

1 Introduction

Dans ce TP, vous allez migrer l'application bancaire développée en JDBC vers une architecture utilisant JPA (Java Persistence API) pour la persistance des données. Vous implémenterez également les patrons DAO (Data Access Object) et DTO (Data Transfer Object) pour structurer votre code de manière professionnelle.

2 Diagramme de la base de données



3 Mise en place de l'environnement

3.1 Étape 1 : Configuration Maven

Ajoutez les dépendances suivantes dans votre fichier `pom.xml` :

```xml
<dependencies>
 <!-- JPA API -->
 <dependency>
  <groupId>jakarta.persistence</groupId>
  <artifactId>jakarta.persistence-api</artifactId>
  <version>3.2.0</version>
 </dependency>

 <!-- Implementation JPA (Hibernate) -->
 <dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>7.1.11.Final</version>
 </dependency>

 <!-- Base de donnees PostgreSQL -->
 <dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.7.8</version>
 </dependency>
</dependencies>
```

3.2 Étape 2 : Configuration JPA

Créez un fichier `persistence.xml` dans `src/main/resources/META-INF/` :

```xml
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="3.1"
    xmlns="https://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
    https://jakarta.ee/xml/ns/persistence/persistence_3_1.xsd">

 <persistence-unit name="banquePU" transaction-type="RESOURCE_LOCAL">
  <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
  <class>com.votreprojet.model.Client</class>
  <class>com.votreprojet.model.Compte</class>
  <class>com.votreprojet.model.Operation</class>
  <class>com.votreprojet.model.Agent</class>
  <class>com.votreprojet.model.Agence</class>

  <properties>
   <property name="jakarta.persistence.jdbc.url"
             value="jdbc:postgresql://localhost:5432/banque_db"/>
   <property name="jakarta.persistence.jdbc.user" value="user"/>
   <property name="jakarta.persistence.jdbc.password" value="password"/>
   <property name="jakarta.persistence.jdbc.driver" value="org.postgresql.Driver"/>
   <property name="hibernate.dialect" value="org.hibernate.dialect.PostgreSQLDialect"/>
   <property name="hibernate.hbm2ddl.auto" value="update"/>
   <property name="hibernate.show_sql" value="true"/>
   <property name="hibernate.format_sql" value="true"/>
  </properties>
 </persistence-unit>
</persistence>
```

4 Entités JPA

4.1 Classe Client

Implémentez la classe `Client` avec les annotations JPA :

```java
package com.votreprojet.model;

import jakarta.persistence.*;
import java.util.Date;
import java.util.List;
@Entity
@Table(name = "Client")
public class Client {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "numClient")
    private int numClient;

    @Column(name = "nomClient", nullable = false)
    private String nomClient;

    @Column(name = "prenomClient", nullable = false)
    private String prenomClient;

    @Column(name = "adClient")
    private String adClient;

    @Column(name = "dateNaissClient")
    @Temporal(TemporalType.DATE)
    private Date dateNaissClient;

    @Column(name = "ageClient")
    private int ageClient;

    @ManyToOne
    @JoinColumn(name = "numAgent", referencedColumnName = "numAgent")
    private Agent agent;

    @ManyToMany
    @JoinTable(
        name = "Compte_Client",
        joinColumns = @JoinColumn(name = "numClient"),
        inverseJoinColumns = @JoinColumn(name = "numCompte")
    )
    private List<Compte> comptes;

    // Constructeurs, getters et setters
}
```

4.2 Classe Compte

Implémentez la classe `Compte` avec les annotations JPA :

```java
package com.votreprojet.model;
import jakarta.persistence.*;
import java.util.List;

@Entity
@Table(name = "Compte")
public class Compte {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    @Column(name = "numCompte")
    private int numCompte;

    @Column(name = "solde")
    private double solde;

    @Column(name = "typeCompte", nullable = false)
    private String typeCompte;

    @OneToMany(mappedBy = "compte", cascade = CascadeType.ALL)
    private List<Operation> operations;

    @ManyToMany(mappedBy = "comptes")
    private List<Client> clients;

    // Constructeurs, getters et setters
}
```

5 Patron DAO avec JPA

5.1 Interface générique DAO

Créez une interface générique pour les DAO :

```
package com.votreprojet.dao;

import java.util.List;
import java.util.Optional;

public interface DAO<T, K> {
    Optional<T> find(K id);
    List<T> findAll();
    void save(T entity);
    void update(T entity);
    void delete(T entity);
}
```

5.2 Implémentation générique DAO

Implémentez une classe générique pour les DAO :

```
package com.votreprojet.dao;

import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityTransaction;
import java.util.List;
import java.util.Optional;

public abstract class AbstractDAO<T, K> implements DAO<T, K> {
    private final Class<T> entityClass;
    protected EntityManager entityManager;

    public AbstractDAO(Class<T> entityClass, EntityManager entityManager) {
        this.entityClass = entityClass;
        this.entityManager = entityManager;
    }

    @Override
    public Optional<T> find(K id) {
        return Optional.ofNullable(entityManager.find(entityClass, id));
    }
```

```
21
22      @Override
23      public List<T> findAll() {
24          return entityManager.createQuery("SELECT e FROM " + entityClass.getSimpleName() + " e",
        entityClass)
25                  .getResultList();
26      }
27
28      @Override
29      public void save(T entity) {
30          EntityTransaction transaction = entityManager.getTransaction();
31          try {
32              transaction.begin();
33              entityManager.persist(entity);
34              transaction.commit();
35          } catch (Exception e) {
36              if (transaction != null && transaction.isActive()) {
37                  transaction.rollback();
38              }
39              throw e;
40          }
41      }
42
43      @Override
44      public void update(T entity) {
45          EntityTransaction transaction = entityManager.getTransaction();
46          try {
47              transaction.begin();
48              entityManager.merge(entity);
49              transaction.commit();
50          } catch (Exception e) {
51              if (transaction != null && transaction.isActive()) {
52                  transaction.rollback();
53              }
54              throw e;
55          }
56      }
57
58      @Override
59      public void delete(T entity) {
60          EntityTransaction transaction = entityManager.getTransaction();
61          try {
62              transaction.begin();
63              entityManager.remove(entityManager.contains(entity) ? entity : entityManager.merge(
        entity));
64              transaction.commit();
65          } catch (Exception e) {
66              if (transaction != null && transaction.isActive()) {
67                  transaction.rollback();
68              }
69              throw e;
70          }
71      }
72  }
```

5.3 DAO Spécifique pour Client

Créez un DAO spécifique pour la classe `Client` :

```
1  package com.votreprojet.dao;
2
3  import com.votreprojet.model.Client;
4  import jakarta.persistence.EntityManager;
5
```

```java
public class ClientDAO extends AbstractDAO<Client, Integer> {
    public ClientDAO(EntityManager entityManager) {
        super(Client.class, entityManager);
    }

    public List<Client> findByAgent(int numAgent) {
        return entityManager.createQuery(
            "SELECT c FROM Client c WHERE c.agent.numAgent = :numAgent", Client.class)
            .setParameter("numAgent", numAgent)
            .getResultList();
    }
}
```

6 Patron DTO

6.1 DTO pour Client

Créez une classe DTO pour `Client` :

```java
package com.votreprojet.dto;

import java.util.Date;

public class ClientDTO {
    private int numClient;
    private String nomClient;
    private String prenomClient;
    private String adClient;
    private Date dateNaissClient;
    private int ageClient;
    private int numAgent;

    // Constructeurs, getters et setters
}
```

6.2 Mapper pour Client

Créez une classe pour mapper les entités aux DTOs :

```java
package com.votreprojet.mapper;

import com.votreprojet.dto.ClientDTO;
import com.votreprojet.model.Client;

public class ClientMapper {
    public static ClientDTO toDTO(Client client) {
        ClientDTO dto = new ClientDTO();
        dto.setNumClient(client.getNumClient());
        dto.setNomClient(client.getNomClient());
        dto.setPrenomClient(client.getPrenomClient());
        dto.setAdClient(client.getAdClient());
        dto.setDateNaissClient(client.getDateNaissClient());
        dto.setAgeClient(client.getAgeClient());
        dto.setNumAgent(client.getAgent() != null ? client.getAgent().getNumAgent() : 0);
        return dto;
    }

    public static Client toEntity(ClientDTO dto) {
        Client client = new Client();
        client.setNumClient(dto.getNumClient());
        client.setNomClient(dto.getNomClient());
```

```
        client.setPrenomClient(dto.getPrenomClient());
        client.setAdClient(dto.getAdClient());
        client.setDateNaissClient(dto.getDateNaissClient());
        client.setAgeClient(dto.getAgeClient());
        // Note: La relation avec Agent doit etre geree separement
        return client;
    }
}
```

7 Service Layer

7.1 Service pour Client

Créez une classe de service pour gérer la logique métier :

```
package com.votreprojet.service;

import com.votreprojet.dao.ClientDAO;
import com.votreprojet.dto.ClientDTO;
import com.votreprojet.mapper.ClientMapper;
import com.votreprojet.model.Client;
import jakarta.persistence.EntityManager;
import java.util.List;
import java.util.stream.Collectors;

public class ClientService {
    private final ClientDAO clientDAO;

    public ClientService(EntityManager entityManager) {
        this.clientDAO = new ClientDAO(entityManager);
    }

    public ClientDTO createClient(ClientDTO clientDTO) {
        Client client = ClientMapper.toEntity(clientDTO);
        clientDAO.save(client);
        return ClientMapper.toDTO(client);
    }

    public ClientDTO getClient(int id) {
        return clientDAO.find(id)
                .map(ClientMapper::toDTO)
                .orElseThrow(() -> new RuntimeException("Client not found"));
    }

    public List<ClientDTO> getAllClients() {
        return clientDAO.findAll().stream()
                .map(ClientMapper::toDTO)
                .collect(Collectors.toList());
    }

    public List<ClientDTO> getClientsByAgent(int numAgent) {
        return clientDAO.findByAgent(numAgent).stream()
                .map(ClientMapper::toDTO)
                .collect(Collectors.toList());
    }
}
```

8 Application principale

8.1 Classe Main

Créez une classe principale pour tester l'application :

```java
package com.votreprojet;

import com.votreprojet.dto.ClientDTO;
import com.votreprojet.service.ClientService;
import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;
import java.util.Date;

public class Main {
    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("banquePU");
        EntityManager em = emf.createEntityManager();

        ClientService clientService = new ClientService(em);

        // Exemple de creation d'un client
        ClientDTO clientDTO = new ClientDTO();
        clientDTO.setNomClient("Dupont");
        clientDTO.setPrenomClient("Jean");
        clientDTO.setAdClient("123 Rue de Paris");
        clientDTO.setDateNaissClient(new Date());
        clientDTO.setAgeClient(30);
        clientDTO.setNumAgent(1);

        ClientDTO createdClient = clientService.createClient(clientDTO);
        System.out.println("Client cree: " + createdClient.getNomClient());

        em.close();
        emf.close();
    }
}
```

9 Tâches à réaliser

Périmètre du TP (3 séances)

On se limite aux entités **Client**, **Compte** et **Operation**. Les entités `Agent` et `Agence` sont facultatives.

9.1 Migration de JDBC à JPA

1. Migrer les classes métier `Client`, `Compte` et `Operation` en entités JPA (avec leurs relations).
2. Implémenter les DAO de ces trois entités en utilisant la classe générique `AbstractDAO`.
3. Créer les DTO de `Client` et `Compte` et leurs mappers.
4. Implémenter un service `BanqueService`.

9.2 Nouvelles fonctionnalités

1. Implémenter une méthode pour transférer de l'argent entre deux comptes (avec gestion des transactions).
2. Créer une méthode pour obtenir le solde total d'un client (somme des soldes de tous ses

comptes).
3. Implémenter une méthode pour obtenir l'historique des opérations d'un compte.

9.3 Tests

1. Écrire des tests unitaires pour le mapper de `Client`.
2. Écrire des tests pour la méthode de transfert du service (cas nominal et solde insuffisant).

9.4 Bonus : observer le problème « N+1 »

1. Vérifier que l'affichage des requêtes SQL générées est activé (`hibernate.show_sql` et `hibernate.format_sq` à `true` dans `persistence.xml`, voir plus haut).
2. Charger tous les clients (`SELECT c FROM Client c`), puis parcourir la liste et afficher le nombre de comptes de chaque client. Combien de requêtes SQL sont envoyées pour N clients ?
3. Réécrire la requête avec `JOIN FETCH` (`SELECT DISTINCT c FROM Client c LEFT JOIN FETCH c.comptes`) et compter à nouveau. Expliquer la différence dans le rapport (lien avec le chapitre 3, Index et optimisation).

10 Livrables

— Un dépôt Git contenant votre code source.
— Un rapport (fichier Markdown ou PDF) expliquant :
  — Les choix techniques que vous avez faits.
  — Les difficultés rencontrées et comment vous les avez résolues.
  — Les différences entre l'approche JDBC et JPA.
— Un fichier Dockerfile et docker-compose.yml mis à jour.
— Un fichier README.md expliquant comment construire et exécuter votre application.

11 Conseils

— Utilisez les transactions pour garantir la cohérence des données.
— Gérez correctement les exceptions JPA.
— Utilisez les logs pour tracer les opérations importantes.
— Testez chaque méthode individuellement avant de passer à la suivante.
— Respectez les principes SOLID dans votre conception.